Sentinel Vehicle Intelligence Prototype Plan

1. Goal and success criteria

Build a one-week prototype demonstrating how a regional edge-to-central system could eventually support **80,000 heterogeneous cameras**. The prototype will discover the current Sentinel catalogue, analyse **two live feeds concurrently**, detect vehicles and plates, match them against a synthetic watchlist, and provide a central command dashboard.

The source architecture comes from :codex-file-citation{path="C:/Users/NITHIS/OneDrive/Desktop/Hackathon/OUR PIPELINE.docx" purpose="source"}, refined to reduce unnecessary prototype complexity.

Prototype must demonstrate

- Dynamic discovery of the Sentinel camera catalogue.
- Two continuously connected live feeds when internet access is available.
- RTSP-over-TCP ingestion with authenticated HLS fallback.
- Vehicle detection, tracking, plate extraction, multi-frame OCR consensus, and deduplication.
- Immediate suspicious-vehicle alerts.
- Thirty-second liveness heartbeats and hourly regional health reports.
- Durable delivery when the central server temporarily fails.
- Camera map, live health, alert queue, route reconstruction, acknowledgement, resolution, false-positive marking, and incident reporting.
- A visibly labelled synthetic alert injector so the complete workflow is always demonstrable.
- A complete offline fallback for presentation-day network failure.
- No persistence of genuine Sentinel images or full plate numbers.

Explicitly out of scope

- Face recognition.
- Real VAHAN, SARTHI, eGujCop, NAFIS, police, or RTO integrations.
- Storing or downloading Sentinel footage.
- Kafka, Redis Cluster, Kubernetes, TensorRT, or C++ in the first prototype.
- Claiming that the laptop itself processes 80,000 cameras.
- Production-grade multi-region availability or legal/compliance certification.

2. Architecture and technology stack

System flow

`Sentinel catalogue → stream adapters → MediaMTX → regional AI worker → SQLite outbox → FastAPI → PostgreSQL/PostGIS → WebSocket → React dashboard`

Docker Compose will be the primary deployment. Docker Desktop's WSL2 backend and NVIDIA CUDA-on-WSL support make GPU containers feasible on this laptop. A native Windows launch for the AI worker will remain available as an emergency fallback if GPU-container validation fails. [Docker Windows installation](https://docs.docker.com/desktop/setup/install/windows-install/) and [NVIDIA CUDA on WSL](https://docs.nvidia.com/cuda/wsl-user-guide/).

Layer	Selected stack	Responsibility
Deployment	Docker Desktop, WSL2, Docker Compose	Reproducible startup and isolated services
Stream access	FFmpeg, MediaMTX	RTSP/HLS access, codec inspection, H264 normalization and browser playback
Edge AI	Python 3.11, PyAV, OpenCV, Ultralytics YOLOv8n, ByteTrack, FastALPR, ONNX Runtime GPU	Timestamp-aware decoding, vehicle tracking and plate recognition

| Edge durability | SQLite | Watchlist snapshot, event outbox and temporary health state |

| Central API | FastAPI, Pydantic, Uvicorn | Event intake, alert lifecycle, reports and WebSockets |

| Persistence | PostgreSQL, PostGIS, SQLAlchemy, Alembic | Camera registry, sightings, alerts, health and spatial queries |

| Dashboard | React, TypeScript, Vite, MUI, TanStack Query | Operator interface |

| GIS and media | MapLibre GL JS, local GeoJSON, HLS.js | Offline-capable map, routes and live previews |

| Charts | Recharts | Health, throughput and latency summaries |

| Quality | pytest, Vitest, React Testing Library, Playwright, Locust, Ruff, mypy, ESLint | Automated verification |

| Observability | Structured JSON logs, Prometheus-format metrics | Troubleshooting and measured claims |

MediaMTX supports external RTSP/HLS sources and exposes streams through RTSP, HLS and WebRTC. It does not perform codec conversion, so FFmpeg performs H265-to-H264 conversion only when required. [MediaMTX configuration](https://mediamtx.org/docs/references/configuration-file) and [MediaMTX architecture](https://mediamtx.org/docs/features/architecture).

ByteTrack is selected because it is the lightweight tracker intended for fast fixed-camera baselines. [Ultralytics tracking documentation](https://docs.ultralytics.com/modes/track). FastALPR provides an integrated plate detector and OCR pipeline using ONNX models. [FastALPR repository](https://github.com/ankandrew/fast-alpr).

Compose services and profiles

- `database`: PostgreSQL/PostGIS with persistent volume and health check.
- `api`: FastAPI, migrations and central matching service.
- `dashboard`: compiled React application.
- `mediamtx`: local stream gateway with recording disabled.

- `stream-adapter`: two FFmpeg processes selected through configuration.

- `edge-worker`: GPU-enabled AI process handling both selected feeds.

- `demo-simulator`: synthetic sightings, health reports and suspicious event generation.

- `online` profile: starts live Sentinel ingestion.

- `offline` profile: replaces Sentinel and AI input with deterministic simulated events.

- Native fallback: runs the identical edge-worker package on Windows while the remaining services stay containerized.

Public contracts

All payloads carry `schema_version`, UTC timestamps, UUID event IDs and a `source_mode` of `live` or `synthetic`.

Core records:

- `Camera`: camera ID, display name, region, optional coordinates, enabled state and health status.

- `PlateSighting`: event ID, edge ID, camera ID, observation time, stream PTS, track ID, masked plate, plate HMAC, confidence values, model version and quality score.

- `Alert`: sighting ID, watchlist reference, reason, severity, status and acknowledgement/resolution audit fields.

- `HealthReport`: reporting window, feed states, decoded frames, inference frames, sightings, alerts, reconnects, dropped frames, queue depth and latency percentiles.

- `WatchlistSnapshot`: version, generated time and hashed plate entries.

- `SyntheticEvidence`: synthetic-only image reference and SHA-256 integrity hash.

Minimum API:

- `POST /v1/sightings` — idempotent event ingestion.

- `POST /v1/health` — heartbeat and hourly summary ingestion.

- `GET /v1/watchlist/snapshot` — versioned edge watchlist.

- `GET /v1/cameras` — catalogue and health status.

- `GET /v1/alerts` and `PATCH /v1/alerts/{id}` — alert workflow.

- `GET /v1/routes?plate=` — route reconstruction using a transiently computed plate HMAC.

- `GET /v1/reports/hourly` — regional summaries.

- `GET /v1/reports/incidents/{id}` — printable incident report.

- `WS /v1/ws/alerts` — live alert and status updates. FastAPI directly supports WebSocket endpoints and testing. [FastAPI WebSockets](https://fastapi.tiangolo.com/advanced/websockets/).

The demo injector endpoint exists only when `DEMO_MODE=true`; every resulting record and dashboard element is visibly marked synthetic.

3. Complete runtime lifecycle

Startup and camera selection

1. Validate configuration and secrets before starting any feed.

2. Run database migrations and warm up both AI models.

3. Retrieve the current Sentinel catalogue instead of hard-coding camera IDs.

4. Register every discovered camera; cameras without coordinates remain visible in the catalogue but not on the map.

5. Run a transient calibration scan that scores connectivity, frame stability, traffic activity, vehicle size and probable plate visibility.

6. Select the two highest-scoring suitable cameras unless configuration explicitly overrides them.

7. Use approximate, reviewed coordinates for those two demo cameras.

8. Mark all unselected cameras as `UNSCHEDULED`, not incorrectly as offline.

Stream lifecycle

1. Attempt RTSP over TCP.

2. After three unsuccessful attempts within approximately twenty seconds, switch that camera to HLS.

3. Probe codec and resolution.

4. Pass H264 through without re-encoding.

5. Convert browser-incompatible H265 to H264 using NVIDIA hardware encoding when available.

6. Publish normalized feeds to MediaMTX.

7. Decode continuously using PTS rather than frame arrival time.

8. On failure, reconnect after 2, 4, 8, 16 and then 30 seconds.

9. Reset tracker state on PTS discontinuity, scene-loop reset or source replacement.

10. Never enable MediaMTX recording.

Detection lifecycle

1. Continuously decode the stream while targeting approximately five inference frames per second.

2. Adapt between two and eight inference frames per second when processing falls behind; do not disconnect or jump forward through the source.

3. Detect vehicles with YOLOv8n.

4. Track them with ByteTrack so the same vehicle is not repeatedly processed.

5. Score candidate crops by plate size, sharpness, brightness and viewing angle.

6. Run FastALPR only on the best crops from each track.

7. Finalize a plate after two matching high-confidence readings or after five attempts.

8. Start with a configurable 0.85 OCR-consensus threshold and recalibrate it from validation results.

9. Normalize spacing and case, but use exact normalized matching only; fuzzy watchlist matching is disabled to avoid false alerts.

10. Suppress repeated ordinary sightings for the same plate and camera for five minutes.

11. Allow a new alert immediately when the same plate appears at a different camera.

Privacy and matching lifecycle

- Full live plate text exists only briefly in edge memory.

- The worker produces `HMAC-SHA256(normalized_plate, deployment_secret)` and a masked representation.

- Only the HMAC, masked plate and metadata are persisted centrally.

- The synthetic watchlist stores the same HMAC representation.

- Route search accepts a plate, computes the HMAC, performs the query and discards the plaintext.

- Genuine Sentinel frames, crops and clips have zero retention.

- Synthetic alerts may carry synthetic evidence images for demonstrating the evidence interface.

- SHA-256 verifies evidence integrity; HMAC-SHA256 protects plate identifiers. These are separate purposes.

Delivery and central processing

1. The edge worker writes every finalized event to its SQLite outbox before transmission.

2. It posts events to FastAPI using a per-edge credential.

3. FastAPI uses the event UUID as an idempotency key.

4. The central service rechecks the watchlist token.

5. Normal sightings are stored and become route-searchable.

6. A match creates an alert in one transaction and broadcasts it over WebSocket.

7. Failed deliveries remain in the outbox and retry with bounded exponential backoff.

8. After recovery, the outbox drains in chronological order without duplicates.

Operator workflow

1. The map marker changes to an alert state.

2. The alert appears at the top of the queue with camera, time, masked plate, confidence, severity and watchlist reason.

3. The operator opens the alert and may view the local live feed; genuine historical imagery is unavailable by design.

4. The operator chooses `Acknowledge`, `Resolve` or `False positive`.

5. Every status transition is appended to an audit trail.

6. The incident view produces a printable report containing metadata, actions, model version, integrity hashes and a prominent synthetic label where applicable.

7. Searching for a plate plots ordered sightings across the available geocoded cameras.

Routine reporting and health

- Edge liveness heartbeat: every 30 seconds.

- `ONLINE`: heartbeat present and stream/inference healthy.

- `DEGRADED`: reconnecting, excessive frame lag or reduced inference rate.

- `OFFLINE`: no heartbeat for 180 seconds.

- `UNSCHEDULED`: catalogued but not assigned for processing.

- Hourly report: uptime, camera states, sightings, alerts, reconnects, dropped frames, outbox depth and latency.

- Suspicious alerts bypass the hourly schedule and are sent immediately.

4. Development lifecycle and agent workflow

Methodologies

- **Risk-first development:** validate Docker GPU access and Sentinel transport before UI polish.

- **Contract-first development:** freeze event schemas and OpenAPI before parallel subsystem work.

- **Vertical slicing:** establish one synthetic event from source to dashboard before adding AI.

- **Synthetic-first testing:** use deterministic fake plates and events; never depend on a chance live match.

- **Measured iteration:** record model accuracy, latency and load results instead of presenting estimated performance as fact.

- **Privacy and security by design:** metadata minimization, HMAC identifiers, secret isolation and audit trails.

- **Failure-driven testing:** deliberately interrupt feeds, API and database connections.

- **Configuration over hard-coding:** camera IDs, thresholds, reports, retention and protocol preference remain configurable.

Subagents

A maximum of three specialist subagents will run concurrently alongside the primary integrator.

Agent	Ownership	Required handoff
Primary integration and DevOps agent	Repository, contracts, Compose, secrets, CI, integration and final demo	Working vertical slice and release checklist
Streaming and computer-vision agent	Sentinel adapter, FFmpeg, MediaMTX, PyAV, YOLO, ByteTrack, FastALPR and outbox	Measured two-feed benchmark and documented failure behaviour
Central backend and data agent	FastAPI, schemas, authentication, watchlist, PostGIS, reports and WebSockets	OpenAPI tests, migrations and idempotency evidence
Dashboard and GIS agent	Map, health states, alerts, live player, route view and report UI	Playwright-tested operator workflow

| QA, security and performance agent | Starts after a specialist slot frees; integration, chaos, privacy, load and demo rehearsal | Acceptance report with raw test outputs |

Ownership boundaries prevent agents from editing the same subsystem. The primary agent alone changes shared contracts after the initial freeze.

Seven-day build sequence

- **Day 0 or Day 1 morning:** install WSL2 and Docker Desktop, reboot if required, confirm GPU visibility inside a container, confirm FFmpeg hardware codecs, validate one RTSP and one HLS connection, and cache model assets for offline use.
- **Day 1:** initialize the repository, Compose stack, data contracts, database migrations, synthetic event path and minimal dashboard alert.
- **Days 2–3:** build live stream normalization, two-feed AI pipeline, central ingestion, map shell and alert workflow in parallel.
- **Day 4:** integrate the first genuine sighting, HMAC matching, durable outbox, WebSocket updates and health states.
- **Day 5:** add route reconstruction, incident/hourly reports, live preview, synthetic evidence and offline profile.
- **Day 6:** execute accuracy, reconnect, outage, idempotency, privacy and load tests; tune sampling and confidence thresholds.
- **Day 7:** freeze dependencies and model files, rehearse online and offline demonstrations, prepare reset scripts, collect metrics and reserve time for defects.

Implementation stops expanding scope after Day 5; Days 6–7 are reserved for reliability.

5. Data, verification, risks and improvements

Required data and assets

- Sentinel credentials stored only in uncommitted local secrets.

- Current camera catalogue fetched at runtime.

- Approximate coordinates for the two selected cameras, marked as demo data.

- A small Gujarat boundary GeoJSON with recorded source and attribution.

- Synthetic watchlist entries, reasons and severity levels.

- Synthetic normal sightings and one deterministic suspicious route.

- Synthetic evidence image generated specifically for the demo.

- Locally cached model weights and container images for offline startup.

- A legally usable Indian plate validation set, preferably 200–500 labelled crops. It is used for evaluation, not automatic training.

- Organizer clarification before any future retention of real images.

Test and acceptance plan

- Two selected live feeds remain connected and processed for at least 15 minutes.

- Online protocol fallback and reconnect behaviour are demonstrated.

- The synthetic suspicious event reaches the dashboard within three seconds on the local stack.

- Duplicate event submission creates one database record and one alert.

- A five-minute central outage creates no event loss; queued events drain after recovery.

- Restarting the edge worker preserves unsent outbox events.

- H264, H265, nonuniform timestamps and loop discontinuities are tested.

- Alert acknowledgement, resolution and false-positive status survive restart.

- Route reconstruction orders sightings correctly by UTC time.

- Genuine live events contain no stored images or plaintext plate values.

- Secrets are absent from repository history, logs, API responses and browser code.

- Offline mode starts without Sentinel access and demonstrates the complete operator workflow.

- Local load test sustains 50 simulated cameras producing a combined 50 events per second for ten minutes with zero loss and measured latency. This is a prototype benchmark, not proof of 80,000-camera capacity.

- OCR exact-match accuracy and character accuracy are reported from the validation set; no accuracy claim is made until these measurements exist.

Expected weak areas in the first draft

- Indian plate accuracy may be poor for distant, angled, dirty, blurred or night-time plates.
- The browser preview currently stalled during read-only portal inspection, so direct RTSP/HLS testing is a Day 0 gate.
- WSL2, Docker and GPU-container support still require installation and validation on this laptop.
- H265 normalization and AI inference will compete for the same 6 GB GPU.
- Approximate coordinates make GIS behaviour demonstrable but not operationally authoritative.
- The offline simulator proves the central workflow, not computer-vision accuracy.
- The 80,000-camera claim is architectural only; production capacity requires measured event rates, regional sizing and multi-node testing.
- Ultralytics licensing must be reviewed before any government or commercial production deployment.
- Real external watchlists, identity systems, key management, high availability and legal retention policies remain production work.

Improvements deliberately included

- Automatic camera suitability scoring instead of permanently choosing camera IDs.
- Multi-frame OCR voting rather than trusting one frame.
- Dual watchlist verification at edge and central layers.
- Privacy-preserving searchable plate HMACs.
- Clear provenance labels distinguishing live, simulated and injected events.
- A built-in network/API interruption demonstration for showing resilience.

- Versioned event contracts so Kafka can later replace HTTP delivery without changing detection or dashboard records.

- Future production evolution: regional autoscaling, Kafka partitions by region/camera, centralized key management, immutable evidence storage, role-based access, redundant central services and audited government-system adapters.

Locked assumptions

- One primary developer working with Codex and specialist subagents.

- Seven-day prototype schedule.

- Thirty currently discoverable Sentinel cameras; two analysed concurrently.

- Online live mode plus complete offline fallback.

- Metadata-only retention for genuine feeds.

- Approximate coordinates for two cameras.

- Synthetic watchlist and clearly labelled injected alert.

- Docker Compose is primary; only the AI worker has a native fallback.

- Durable HTTP outbox is used now; Kafka is deferred.

- Single local operator account for the prototype.

- All timestamps are stored in UTC and displayed in IST.

- Implementation begins only after this design is accepted; later changes are handled as explicit scope revisions.